

Методичка: связи нескольких таблиц в PostgreSQL

1. Зачем вообще связывать таблицы

В базе данных редко хранят всё в одной огромной таблице. Обычно данные разделяют на несколько таблиц.

Например, плохой вариант:

id	user_name	login	country_name	country_code
1	Ivan Petrov	ivan_login	Russia	RU
2	Anna Smirnova	anna_sm	Russia	RU
3	Oleg Ivanov	oleg_777	Belarus	BY

Проблема: название страны повторяется много раз.

Лучше разделить данные:

users

id	name
1	Ivan Petrov
2	Anna Smirnova

countries

id	name
1	Russia
2	Belarus

А потом связать эти таблицы через специальное поле.

2. Основные понятия

Primary Key — первичный ключ

Primary Key, или `PRIMARY KEY`, — это главный идентификатор строки.

Например:

```
id SERIAL PRIMARY KEY
```

Это значит:

`id` — уникальный номер строки

Пример:

id	name
1	Ivan Petrov
2	Anna Smirnova

У каждой строки свой `id`.

Foreign Key — внешний ключ

Foreign Key, или `FOREIGN KEY`, — это поле, которое ссылается на `id` из другой таблицы.

Например:

```
users.country_id —————> countries.id
```

То есть поле `country_id` в таблице `users` хранит номер страны из таблицы `countries`.

3. Типы связей между таблицами

В PostgreSQL чаще всего используют 3 вида связей:

1. Один к одному
2. Один ко многим
3. Многие ко многим

4. Связь один к одному

Идея связи

Связь **один к одному** означает:

Одна запись из первой таблицы связана только с одной записью из второй т

Например:

Один пользователь имеет одну карточку с дополнительной информацией.

Пример таблиц

```
users
id
name
login
password
users_ext_info_id
```

```
users_ext_info
id
credit_card
adress
info
```

Схема связи:

users.users_ext_info_id → users_ext_info.id

Визуализация с данными

users				
id	name	login	password	users_ext_info_id
1	Ivan Petrov	ivan_login	qwerty123	1
2	Anna Smirnova	anna_sm	pass456	2
3	Oleg Ivanov	oleg_777	olegpass	3

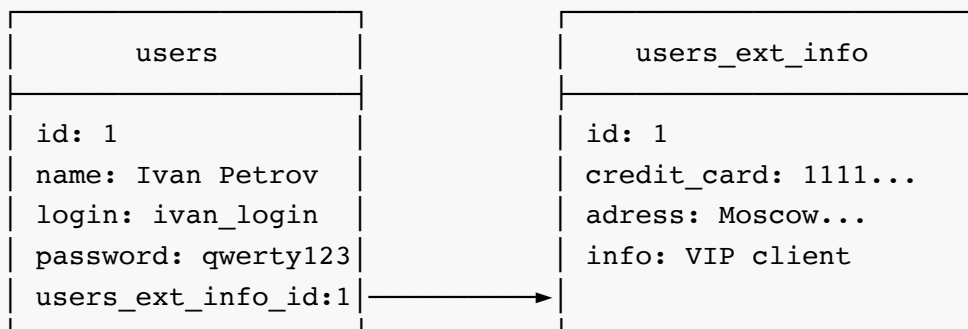
users.users_ext_info_id

users_ext_info			
id	credit_card	adress	info
1	1111 2222 3333	Moscow, Lenina 10	VIP client
2	4444 5555 6666	Kazan, Mira 25	New user
3	7777 8888 9999	Minsk, Central 7	Regular user

Как читать связь

У Ivan Petrov users_ext_info_id = 1

Значит, нужно найти строку в users_ext_info, где id = 1.



CREATE TABLE для PostgreSQL

```
CREATE TABLE users_ext_info (  
    id SERIAL PRIMARY KEY,  
    credit_card VARCHAR(50),  
    adress VARCHAR(255),  
    info TEXT  
);  
  
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    login VARCHAR(100),  
    password VARCHAR(100),  
    users_ext_info_id INT UNIQUE,  
    FOREIGN KEY (users_ext_info_id) REFERENCES users_ext_info(id)  
);
```

Обрати внимание на `UNIQUE` :

```
users_ext_info_id INT UNIQUE
```

Он нужен, чтобы одна дополнительная карточка не могла принадлежать сразу нескольким пользователям.

INSERT с тестовыми данными

```
INSERT INTO users_ext_info (credit_card, adress, info)  
VALUES  
( '1111 2222 3333', 'Moscow, Lenina 10', 'VIP client'),  
( '4444 5555 6666', 'Kazan, Mira 25', 'New user'),  
( '7777 8888 9999', 'Minsk, Central 7', 'Regular user');  
  
INSERT INTO users (name, login, password, users_ext_info_id)  
VALUES  
( 'Ivan Petrov', 'ivan_login', 'qwerty123', 1),  
( 'Anna Smirnova', 'anna_sm', 'pass456', 2),  
( 'Oleg Ivanov', 'oleg_777', 'olegpass', 3);
```

В реальных проектах пароль нельзя хранить открытым текстом. Его нужно хешировать. Но для учебного примера поле `password` можно оставить обычной строкой.

SELECT с JOIN

```
SELECT
    users.name,
    users.login,
    users_ext_info.credit_card,
    users_ext_info.adress,
    users_ext_info.info
FROM users
JOIN users_ext_info
ON users.users_ext_info_id = users_ext_info.id;
```

Результат будет примерно такой:

name	login	credit_card	adress	info
Ivan Petrov	ivan_login	1111 2222 3333	Moscow, Lenina 10	VIP c
Anna Smirnova	anna_sm	4444 5555 6666	Kazan, Mira 25	New t
Oleg Ivanov	oleg_777	7777 8888 9999	Minsk, Central 7	Regu

5. Связь один ко многим

Идея связи

Связь **один ко многим** означает:

Одна запись из первой таблицы может быть связана со многими записями из

Например:

Одна страна может быть у многих пользователей.

Пример таблиц

```
users
id
name
login
password
country_id
```

```
countries
id
name
```

Схема связи:

```
users.country_id  —————>  countries.id
```

Визуализация с данными

users				
id	name	login	password	country_id
1	Ivan Petrov	ivan_login	qwerty123	1
2	Anna Smirnova	anna_sm	pass456	2
3	Oleg Ivanov	oleg_777	olegpass	1
4	Maria Sokolova	maria_s	123456	3

users.country_id

countries	
id	name
1	Russia
2	Kazakhstan
3	Belarus

Как читать связь

country_id = 1 $\longrightarrow$ countries.id = 1 $\longrightarrow$ Russia

Значит:

Ivan Petrov ЖИВЁТ В Russia
Oleg Ivanov ЖИВЁТ В Russia

Одна страна связана с несколькими пользователями.

Russia:
├─ Ivan Petrov
└─ Oleg Ivanov

Kazakhstan:
└─ Anna Smirnova

Belarus:
└─ Maria Sokolova

CREATE TABLE для PostgreSQL

```
CREATE TABLE countries (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    login VARCHAR(100),  
    password VARCHAR(100),  
    country_id INT,  
    FOREIGN KEY (country_id) REFERENCES countries(id)  
);
```

INSERT с тестовыми данными


```
INSERT INTO countries (name)
VALUES
('Russia'),
('Kazakhstan'),
('Belarus');

INSERT INTO users (name, login, password, country_id)
VALUES
('Ivan Petrov', 'ivan_login', 'qwerty123', 1),
('Anna Smirnova', 'anna_sm', 'pass456', 2),
('Oleg Ivanov', 'oleg_777', 'olegpass', 1),
('Maria Sokolova', 'maria_s', '123456', 3);
```

SELECT с JOIN

```
SELECT
    users.name,
    users.login,
    countries.name AS country
FROM users
JOIN countries
ON users.country_id = countries.id;
```

Результат:

name	login	country
Ivan Petrov	ivan_login	Russia
Anna Smirnova	anna_sm	Kazakhstan
Oleg Ivanov	oleg_777	Russia
Maria Sokolova	maria_s	Belarus

6. СВЯЗЬ МНОГИЕ КО МНОГИМ

Идея связи

Связь **многие ко многим** означает:

Одна запись из первой таблицы может быть связана со многими записями из

И наоборот:

Одна запись из второй таблицы может быть связана со многими записями из

Например:

Один автор может написать много книг.
Одна книга может быть написана несколькими авторами.

Пример таблиц

```
authors
id
name
age
rating

books_authors
id
author_id
book_id

books
id
name
price
```

Здесь появляется специальная промежуточная таблица:

```
books_authors
```

Она хранит не самих авторов и не сами книги, а связи между ними.

Схема связи

```
authors.id  ← books_authors.author_id
books.id    ← books_authors.book_id
```

Общая схема:



Визуализация с данными

authors			
id	name	age	rating
1	George Orwell	46	9.5
2	J.K. Rowling	58	9.2
3	Stephen King	76	8.9
4	Peter Straub	79	8.4



books_authors.author_id

books_authors		
id	author_id	book_id
1	1	1
2	1	2
3	2	3
4	3	4
5	3	5
6	4	5



books_authors.book_id

books		
id	name	price
1	1984	750
2	Animal Farm	500
3	Harry Potter	1200
4	The Shining	900
5	The Talisman	1100

Как читать связь

Строка в books_authors :

```
author_id = 1
book_id = 1
```

означает:

Автор с id = 1 написал книгу с id = 1.

То есть:

George Orwell написал 1984.

Строка:

```
author_id = 3
book_id = 5
```

означает:

Stephen King написал The Talisman.

Строка:

```
author_id = 4
book_id = 5
```

означает:

Peter Straub тоже написал The Talisman.

Значит, у книги `The Talisman` два автора.

Визуализация по смыслу

George Orwell:

- └─ 1984
- └─ Animal Farm

J.K. Rowling:

- └─ Harry Potter

Stephen King:

- └─ The Shining
- └─ The Talisman

Peter Straub:

- └─ The Talisman

А если смотреть со стороны книг:

1984:

- └─ George Orwell

Animal Farm:

- └─ George Orwell

Harry Potter:

- └─ J.K. Rowling

The Shining:

- └─ Stephen King

The Talisman:

- └─ Stephen King
- └─ Peter Straub

CREATE TABLE для PostgreSQL

```
CREATE TABLE authors (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    age INT,  
    rating NUMERIC(3, 1)  
);  
  
CREATE TABLE books (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(150) NOT NULL,  
    price INT  
);  
  
CREATE TABLE books_authors (  
    id SERIAL PRIMARY KEY,  
    author_id INT NOT NULL,  
    book_id INT NOT NULL,  
  
    FOREIGN KEY (author_id) REFERENCES authors(id),  
    FOREIGN KEY (book_id) REFERENCES books(id)  
);
```

INSERT с тестовыми данными

```
INSERT INTO authors (name, age, rating)
VALUES
('George Orwell', 46, 9.5),
('J.K. Rowling', 58, 9.2),
('Stephen King', 76, 8.9),
('Peter Straub', 79, 8.4);

INSERT INTO books (name, price)
VALUES
('1984', 750),
('Animal Farm', 500),
('Harry Potter', 1200),
('The Shining', 900),
('The Talisman', 1100);

INSERT INTO books_authors (author_id, book_id)
VALUES
(1, 1),
(1, 2),
(2, 3),
(3, 4),
(3, 5),
(4, 5);
```

SELECT с JOIN

Чтобы получить книги и их авторов, нужно соединить сразу 3 таблицы:

```
SELECT
    books.name AS book,
    books.price,
    authors.name AS author,
    authors.rating
FROM books
JOIN books_authors
ON books.id = books_authors.book_id
JOIN authors
ON authors.id = books_authors.author_id;
```

Результат:

book	price	author	rating
1984	750	George Orwell	9.5
Animal Farm	500	George Orwell	9.5
Harry Potter	1200	J.K. Rowling	9.2
The Shining	900	Stephen King	8.9
The Talisman	1100	Stephen King	8.9
The Talisman	1100	Peter Straub	8.4

7. Почему для многие ко многим нужна промежуточная таблица

Без таблицы `books_authors` пришлось бы хранить авторов прямо в таблице `books`.

Например:

id	book	authors
5	The Talisman	Stephen King, Peter Straub

Это плохой вариант, потому что:

1. Сложно искать книги конкретного автора.
2. Сложно сортировать и фильтровать авторов.
3. Сложно обновлять данные.
4. Нельзя нормально настроить FOREIGN KEY.

Правильный вариант:

authors		books_authors		books
Stephen King	→	The Talisman	←	Peter Straub

То есть связь хранится отдельными строками.

8. Как определить тип связи

Один к одному

Вопрос:

Может ли одна запись из первой таблицы иметь несколько записей из второй

Если нет, скорее всего это связь **один к одному**.

Пример:

Один пользователь — одна карточка дополнительной информации.

Один ко многим

Вопрос:

Может ли одна запись из первой таблицы быть связана с несколькими записями

Если да, но обратная связь только одна, это **один ко многим**.

Пример:

Одна страна — много пользователей.
Но один пользователь обычно относится к одной стране.

Многие ко многим

Вопрос:

Может ли первая таблица иметь много записей из второй,
и вторая таблица иметь много записей из первой?

Если да, это **многие ко многим**.

Пример:

Один автор — много книг.
Одна книга — много авторов.

9. Главное правило

Тип связи определяется не количеством данных, которые сейчас есть в таблице, а логикой предметной области.

Например, сейчас у книги может быть только один автор:

Harry Potter:
└─ J.K. Rowling

Но если по логике системы книга может иметь нескольких авторов, значит нужно проектировать связь **многие ко многим**.

10. Мини-шпаргалка

Один к одному:

table_1.id — table_2.table_1_id

Пример:

users — users_ext_info

Один ко многим:

one_table.id ← many_table.one_table_id

Пример:

countries.id ← users.country_id

Многие ко многим:

table_1 ← middle_table → table_2

Пример:

authors ← books_authors → books

11. Практические задания

Задание 1

Есть таблицы:

```
students
id
name
age
student_card_id

student_cards
id
number
created_at
```

Определить тип связи.

Ответ:

Один к одному

Потому что у одного ученика одна студенческая карточка.

Задание 2

Есть таблицы:

```
groups
id
name

students
id
name
group_id
```

Определить тип связи.

Ответ:

Один ко многим

Потому что в одной группе может быть много учеников.

Задание 3

Есть таблицы:

```
students
id
name

courses
id
title

students_courses
id
student_id
course_id
```

Определить тип связи.

Ответ:

Многие ко многим

Потому что один ученик может ходить на несколько курсов, и на одном курсе может быть много учеников.

12. Итог

Связи между таблицами нужны, чтобы данные не дублировались и база была удобной.

Один к одному:

одна запись связана только с одной записью.

Один ко многим:

одна запись связана со многими записями.

Многие ко многим:

много записей с одной стороны связаны со многими записями с другой сторо

Главные инструменты для связей в PostgreSQL:

PRIMARY KEY

FOREIGN KEY

JOIN

`PRIMARY KEY` создаёт главный идентификатор строки. `FOREIGN KEY` создаёт ссылку на другую таблицу. `JOIN` позволяет получить связанные данные из нескольких таблиц.